

# Separation

Dror Fried\*, Avital Parasha\*, Lucas M. Tabajara†

\*The Open University of Israel, Israel

Email: dfried@openu.ac.il, avitalpresearch@gmail.com

†TBD

Email: <lucas.email@domain>

**Abstract—**

## I. INTRODUCTION

To demonstrate the scalability and practical applicability of our synthesis framework, we introduce a realistic hardware case study from the VLSI domain. Driven by Moore’s law, continuous technology scaling has enabled the integration of a vastly growing number of components onto a single die. As the density of functional blocks increases, the underlying communication infrastructure connecting them has become as critical as the computing units themselves. To address these communication demands efficiently, the Network-on-Chip (NoC) paradigm was introduced by Dally and Towles to connect extensive many-core and System-on-a-Chip (SoC) architectures [1]. By replacing ad-hoc global wiring with structured networks, NoCs provide essential modularity and well-controlled electrical parameters [1]. Today, hardened NoCs are the integrated backbone of highly complex SoC architectures and multi-die commercial devices [2]. However, this architectural power comes at the cost of immense logical complexity; a single NoC router can contain over 1,000 state variables, rendering traditional formal analysis highly susceptible to the state-explosion problem [3]. To operate reliably, NoCs must support strict Quality-of-Service (QoS) requirements, priority-driven arbitration, deadlock-preventing virtual channel allocation, and diverse hardware protocols. In this work, we formally capture these core routing and arbitration behaviors using the class of separated LTL specifications addressed in our framework.

## II. RELATED WORK

The deployment of formal methods and synthesis techniques has a strong precedent within the domain of on-chip communication and Network-on-Chip (NoC) systems. Initial breakthroughs in correct-by-construction reactive synthesis effectively addressed fundamental communication standards, notably the ARM AMBA AHB bus protocol, using General Reactivity of rank 1 (GR(1)) games [4], [5]. Such protocols exemplify the standard interfaces commonly deployed at the ingress and egress ports of modern NoCs [4]. In a separate vein, formal verification methodologies have been rigorously applied to the NoC Quality-of-Service (QoS) space; techniques such

as bounded model checking and compositional inductive proofs have been utilized to ensure critical performance targets, including end-to-end latency bounds and optimal buffer sizing [6], [7]. However, verifying latency bounds often requires unrolling transition relations for hundreds or thousands of clock cycles, pushing NoC verification beyond the capacity of state-of-the-art model checkers and requiring complex compositional proofs [7]. Historically, general LTL synthesis was viewed as hopelessly intractable due to its doubly-exponential time complexity [4]. While prior reactive synthesis methodologies for on-chip protocols successfully bypassed this by leaning heavily on GR(1) frameworks [5], GR(1) suffers from limited expressivity and cannot represent full Reactivity (Streett) conditions [4]. Our research advances the field by exploring a dedicated fragment of GR(K). This chosen fragment retains high expressiveness and scalability, smoothly accommodating intuitive, sophisticated NoC routing scenarios that would be either highly impractical or mathematically impossible to express strictly within a GR(1) construct.

## III. PRELIMINARIES

### A. General Definitions.

Given a set of Boolean variables  $\mathcal{V}$ , a *state over  $\mathcal{V}$*  is an assignment  $s$  to the variables in  $\mathcal{V}$ . We describe  $s$  as the subset of  $\mathcal{V}$  that is assigned **True** in  $s$ . The set of *primed variables of  $\mathcal{V}$*  is  $\mathcal{V}' = \{v' \mid v \in \mathcal{V}\}$ . Then  $s' = \{v' \mid v \in s\}$  is the primed state  $s'$  over  $\mathcal{V}'$ . An *assertion over  $\mathcal{V}$*  is a Boolean formula over variables  $\mathcal{V}$ . A state  $s$  satisfies an assertion  $\rho$  over the same variables, denoted  $s \models \rho$ , if  $\rho$  evaluates to **True** by assigning *true* exactly to the elements of  $s$ . We define the *projection* of a state  $s$  on a subset  $\mathcal{U} \subseteq \mathcal{V}$  as denoted by  $s|_{\mathcal{U}} = s \cap \mathcal{U}$ . We extend the notion of projection to a set of states  $S \subseteq 2^{\mathcal{V}}$  by defining  $S|_{\mathcal{U}} = \{s|_{\mathcal{U}} \mid s \in S\}$ .

Our specification is a special form of *Linear Temporal Logic* (LTL). LTL [8] extends propositional logic with infinite-horizon temporal operators. The syntax of an LTL formula over a finite set of Boolean variables  $\mathcal{V}$  is defined as follows:  $\varphi ::= v \in \mathcal{V} \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \mathbf{X}\varphi \mid \varphi\mathbf{U}\varphi \mid \mathbf{F}\varphi \mid \mathbf{G}\varphi$ . Here **X** (Next), **U** (Until), **F** (Eventually), **G** (Always) are temporal operators. The semantics of LTL can be found in [9, Chapter 5].

We model the adapters as transducers. A *transducer* is a deterministic finite-state machine with no accepting

states, but with additional output alphabet and an additional function from the set of states to the output alphabet. A formal definition of a transducer can be found in [10], but is not required for this paper.

The algorithms developed in this paper are symbolic, i.e. manipulate implicit representations of sets of states. To this end, we use *Binary Decision Diagrams (BDDs)* [11], [12] to represent assertions. For a BDD  $B$  and sets of variables  $\mathcal{V}_1, \dots, \mathcal{V}_n$ , we write  $B(\mathcal{V}_1, \dots, \mathcal{V}_n)$  to denote that  $B$  represents an assertion over  $\mathcal{V}_1 \cup \dots \cup \mathcal{V}_n$ . For a state  $s$  over  $\mathcal{V}$ , we write  $s \models B(\mathcal{V})$  to denote that the assertion that  $B$  represents is satisfied by the state  $s$ . BDDs support several *symbolic operations*: conjunction ( $\wedge$ ), disjunction ( $\vee$ ), negation ( $\neg$ ), and extraction of variables using the  $\exists$  and  $\forall$  operators. We measure time complexity of a symbolic algorithm by a worst case #symbolic-operations it performs. A discussion on a rigorous treatment of BDD operations can be found in Appendix III.

### B. Game Structures and Games.

We follow the notations of [13]. A game structure  $GS = (\mathcal{I}, \mathcal{O}, \theta_{\mathcal{I}}, \theta_{\mathcal{O}}, \rho_{\mathcal{I}}, \rho_{\mathcal{O}})$  defines a turn-based interaction between an *environment* and a *system* players. The input variables  $\mathcal{I}$  and output variables  $\mathcal{O}$  are two disjoint sets of Boolean variables that are controlled by the environment and system, respectively. The environment's *initial assumption*  $\theta_{\mathcal{I}}$  is an assertion over  $\mathcal{I}$ , and the system's *initial guarantee*  $\theta_{\mathcal{O}}$  is an assertion over  $\mathcal{I} \cup \mathcal{O}$ . The environment's *safety assumption*  $\rho_{\mathcal{I}}$  is an assertion over  $\mathcal{I} \cup \mathcal{O} \cup \mathcal{I}'$ , where the interpretation of  $(i_0, o_0, i'_1) \models \rho_{\mathcal{I}}$  is that from state  $(i_0, o_0)$  the environment can assign  $i_1$  to the input variables. W.l.o.g, we assume that  $\rho_{\mathcal{I}}$  is deadlock free, i.e., for all  $(i_0, o_0)$  there exists an  $i_1$  s.t.  $(i_0, o_0, i'_1) \models \rho_{\mathcal{I}}$ . Similarly, the system's *safety guarantee*  $\rho_{\mathcal{O}}$  is an assertion over  $\mathcal{I} \cup \mathcal{O} \cup \mathcal{I}' \cup \mathcal{O}'$ , where the interpretation of  $(i_0, o_0, i'_1, o'_1) \models \rho_{\mathcal{O}}$  is that from state  $(i_0, o_0)$  when the environment assigns  $i_1$  to the input variables, the system can assign  $o_1$  to the output variables. Again, w.l.o.g, we assume that  $\rho_{\mathcal{O}}$  is deadlock free, i.e., for all  $(i_0, o_0, i'_1)$  there exists an  $o_1$  s.t.  $(i_0, o_0, i'_1, o'_1) \models \rho_{\mathcal{O}}$ .

A play over  $GS$  progresses by the players taking turns to assign values to their own variables ad infinitum, where the players must satisfy the initial conditions at the start and the safety conditions thereafter. Formally, a *play*  $\pi = s_0, s_1, \dots$  is an infinite sequence of states over  $\mathcal{I} \cup \mathcal{O}$  such that  $s_0 \models \theta_{\mathcal{I}} \wedge \theta_{\mathcal{O}}$  and  $(s_j, s'_{j+1}) \models \rho_{\mathcal{I}} \wedge \rho_{\mathcal{O}}$  for all  $j \geq 0$ . A *play prefix* is either a play or a finite sequence of states that can be extended to a play. Then a *strategy* is a function  $f : (2^{\mathcal{I} \cup \mathcal{O}})^+ \times 2^{\mathcal{I}} \rightarrow 2^{\mathcal{O}}$  such that if  $s_0, \dots, s_m$  is a play prefix,  $(s_m, i') \models \rho_{\mathcal{I}}$  and  $f(s_0, \dots, s_m, i) = o$ , then  $(s_m, i', o') \models \rho_{\mathcal{O}}$ . Intuitively, a strategy directs the system on what to assign to the output variables, depending on the history of a play and the most recent assignment by the environment to the input variables. A play prefix is said to be *consistent with a strategy*  $f$  if for all states  $s_j = (i_j, o_j)$  in that prefix,  $f(s_0, \dots, s_{j-1}, i_j) = o_j$  for all  $j \geq 0$ . A

strategy is *memoryless* if it only depends on the last state and the most recent assignment to the input variables. Formally, a *memoryless strategy* is a function  $f : (2^{\mathcal{I} \cup \mathcal{O}}) \times 2^{\mathcal{I}} \rightarrow 2^{\mathcal{O}}$  such that if  $(s_m, i') \models \rho_{\mathcal{I}}$  and  $f(s_m, i') = o$ , then  $(s_m, i', o') \models \rho_{\mathcal{O}}$ .

A *game* is a tuple  $(GS, \varphi)$  where  $GS$  is a game structure over inputs  $\mathcal{I}$  and outputs  $\mathcal{O}$  and  $\varphi$  is an LTL formula over  $\mathcal{I} \cup \mathcal{O}$  called a *winning condition*. A play  $\pi$  is *winning* for the system if  $\pi \models \varphi$ . A strategy  $f$  *wins from state*  $s$  if every play  $\pi$  from  $s$  that is consistent with  $f$  is winning for the system. A strategy  $f$  *wins from*  $S$ , where  $S$  is an assertion over  $\mathcal{I} \cup \mathcal{O}$ , if it wins from every state  $s \models S$ . The *winning region* of the system is the set of states from which it has a winning strategy. A strategy  $f$  is *winning* if for every state  $i \models \theta_{\mathcal{I}}$  there exists a state  $o \in 2^{\mathcal{O}}$  such that  $(i, o) \models \theta_{\mathcal{O}}$  and  $f$  win from  $(i, o)$ . In this paper, we are leaning on the results of [14] that look at the following games that are defined under the following winning conditions.

- *Reachability games*:  $F(\varphi)$  where  $\varphi$  is an assertion over  $\mathcal{I} \cup \mathcal{O}$ .
- *Safety games*:  $G(\varphi)$  where  $\varphi$  is an assertion over  $\mathcal{I} \cup \mathcal{O}$ .
- *Büchi games*:  $GF(\varphi)$  where  $\varphi$  is an assertion over  $\mathcal{I} \cup \mathcal{O}$ .
- *GR(k) games*:  $\bigwedge_{l=1}^k (\bigwedge_{i=1}^{n_l} GF(\varphi_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} GF(\psi_{l,j}))$  where all  $\varphi_{l,i}$  and  $\psi_{l,j}$  are assertions over  $\mathcal{I} \cup \mathcal{O}$ .

We provide polynomial-time solutions for "(Settle-Sustain)" and "(Sustain-Settle)" cases. For case "(Settle-Settle)", we explore multiple directions and present the challenges and interesting scenarios we encountered.

Given a game  $(GS, \varphi)$ , *realizability* is the problem of deciding whether a winning strategy for the system exists, and *synthesis* is the problem of constructing a winning strategy if one exists. We note that a realizability check can be reduced to the identification of the winning region,  $W$ : A winning strategy exists iff for all  $i \models \theta_{\mathcal{I}}$  there exists  $o \in 2^{\mathcal{O}}$  such that  $(i, o) \models \theta_{\mathcal{O}}$  and  $(i, o) \in W$ . Hence, the synthesis problem can be solved by constructing a strategy that wins from  $W$ .

### C. Game Graphs and Weak Büchi Games.

The *game graph* for a game structure  $GS$  is the directed graph  $(V, E)$  with vertices  $V = 2^{\mathcal{I} \cup \mathcal{O}}$  and edges  $E = \{(s, t) \mid (s, t') \models \rho_{\mathcal{I}} \wedge \rho_{\mathcal{O}}\}$ . Intuitively, vertices are states over  $\mathcal{I}$  and  $\mathcal{O}$ , and edges represent valid transitions between states according to the safety conditions. The game graph can be useful for analyzing the structural properties of a game structure via graph-theoretical properties.

A *finite path* in a directed graph  $(V, E)$  is a sequence  $v_0, \dots, v_n \in V^+$  such that  $(v_j, v_{j+1}) \in E$  for all  $0 \leq j < n$ . An *infinite path*  $v_0, v_1, \dots \in V^\omega$  is similarly defined. A vertex  $u$  is said to be *reachable* from another vertex  $v$  if there is a finite path from  $v$  to  $u$ . A *strongly connected component* (SCC) of a directed graph  $(V, E)$  is a maximal set of vertices within which every vertex is reachable from every other vertex. It is well known that SCCs partition the set of vertices of a directed graph, and that the set

of SCCs is partially ordered with respect to reachability. Also note that every infinite path ultimately stays in an SCC.

Let  $(GS, GF\varphi)$  be a game with a Büchi winning condition, and let  $S_0 \dots, S_m$  be the set of SCCs that partition the game graph of  $GS$ . We say that  $(GS, GF\varphi)$  is a *weak Büchi game* if, given the set  $\mathcal{F}$  of states that satisfy the assertion  $\varphi$ , for every SCC  $S_i$ , either  $S_i \subseteq \mathcal{F}$  or  $S_i \cap \mathcal{F} = \emptyset$ . Thus, the SCCs of a weak Büchi game are either *accepting components*, meaning all of its states are contained in  $\mathcal{F}$ , or *non-accepting components*, meaning none of its states is present in  $\mathcal{F}$ . As a consequence, a play in a weak Büchi game is winning for the system if the play ultimately never exits an accepting component. Similarly, a strategy is winning for the system if it can guarantee that every play will ultimately remain inside an accepting component.

#### D. Nocs

### IV. SEPARATED SPECIFICATIONS IN NOC

The synthesis of reactive systems from Linear Temporal Logic (LTL) specifications [15] hinges on the delicate interplay between environment assumptions and system guarantees. In the context of separated LTL synthesis, we often categorize specifications into four fundamental archetypes based on the frequency and persistence of signals. These patterns, defined by the interaction of safety and liveness operators (specifically  $G$  and  $F$ ), provide a blueprint for how a system should respond to its surroundings [16].

The motivation for identifying these patterns lies in the practical constraints of real-world engineering and the complexity of the synthesis game. Not all reactive behaviors are computationally equal; some require strict fairness, while others prioritize long-term stability. By partitioning the specification space, we can leverage specialized algorithms—such as those used for Generalized Reactivity of rank 1 (GR(1))—to mitigate the state-explosion problem [17]

*a) Type 1: Persistent Interaction ( $GF\phi \rightarrow GF\psi$ ):* This pattern represents the classic fairness constraint, often referred to as a "strong fairness" or "justice" condition. It is motivated by scenarios where the environment provides a stream of persistent requests that necessitate persistent responses. If the environment fulfills its contract by requesting infinitely often, the system must ensure that no requester is starved.

- *Intuition:* Fairness is the critical metric here. This is essential for services like distributed systems and resource allocation, such as **room service** or **email delivery systems**, where every incoming request must eventually be processed to ensure system utility [18]

*b) Type 2: Adaptive Convergence ( $GF\phi \rightarrow FG\psi$ ):* In this archetype, the system is expected to learn or adapt from persistent environmental feedback until it reaches an optimal, stable configuration. Even if the input remains

dynamic, the system aims to "solve" the environment and settle into a desired state.

- *Intuition:* The goal is convergence. This mirrors the behavior of **smart thermostats**, **self-stabilizing systems** or **adaptive spam filters** that, after a period of training or exposure to a dynamic environment and data, settle and stabilize into a stable output boundary.

*c) Type 3: Environmental Quiescence ( $FG\phi \rightarrow FG\psi$ ):* This pattern describes systems that value stability in response to a stabilizing environment. It is motivated by the "settling down" principle: if the environment eventually becomes constant (quiescent), the system should likewise cease unnecessary transitions and reach a steady state.

- *Intuition:* This is particularly useful for energy-saving or maintenance modes. For instance, a system entering **night mode** or a server reaching **weekend quiet** after a heavy data migration assumes the environment has stabilized and responds by reducing its own activity.

*d) Type 4: Operational Vitality ( $FG\phi \rightarrow GF\psi$ ):* Unlike the previous types, Type 4 requires the system to remain "alive" and active even if the environment has reached a steady state. It is motivated by the need for periodic maintenance and liveness proofs that are independent of external stimuli.

- *Intuition:* The system must provide periodic proofs of life. This is standard in **watchdog timers**, **heart-beat monitors**, **automated backups** and **automated diagnostic routines** that must trigger periodically to satisfy safety-critical liveness properties [19].

By separating specifications into these types, we can apply specialized synthesis algorithms that leverage the specific temporal structures of each pattern, leading to more efficient controller generation and clearer system requirements.

### V. SEPARATED GAMES

A separated GR( $k$ ) differentiates from a regular GR( $k$ ) game in a separation between input and output variables in both the game structure and winning condition. We show in later sections that the separation of variables leads to algorithmic benefits to the synthesis problem. Formally,

**Definition 1.** A game structure  $GS = (\mathcal{I}, \mathcal{O}, \theta_{\mathcal{I}}, \theta_{\mathcal{O}}, \rho_{\mathcal{I}}, \rho_{\mathcal{O}})$  separates variables over input variables  $\mathcal{I}$  and output variables  $\mathcal{O}$  if:

- The environment's initial assumption  $\theta_{\mathcal{I}}$  is an assertion over  $\mathcal{I}$  only.
- The system's initial guarantees  $\theta_{\mathcal{O}}$  is an assertion over  $\mathcal{O}$  only.
- The environment's safety assumption  $\rho_{\mathcal{I}}$  is an assertion over  $\mathcal{I} \cup \mathcal{I}'$  only.
- The system's safety guarantee  $\rho_{\mathcal{O}}$  is an assertion over  $\mathcal{O} \cup \mathcal{O}'$  only.

The interpretation of a game structure which separates variables is that the underlying game graph  $(V, E)$  is the product of two distinct directed graphs over disjoint sets of variables:  $G_{\mathcal{I}}$  over the variables  $\mathcal{I} \cup \mathcal{I}'$ , and  $G_{\mathcal{O}}$  over the variables  $\mathcal{O} \cup \mathcal{O}'$ . For  $\mathcal{J} \in \{\mathcal{I}, \mathcal{O}\}$ , the vertices of  $G_{\mathcal{J}}$  correspond to states over  $\mathcal{J}$  and there is an edge between states  $s$  and  $t$  if  $(s, t') \models \rho_{\mathcal{J}}$ .

Next, the notion of separation of variables over  $\mathcal{I} \cup \mathcal{O}$  extends to games with winning conditions as follows:

- *Recurrence to Recurrence (R2R) Games (also called separated GR(k) games)* :  $\bigwedge_{l=1}^k (\bigwedge_{i=1}^{n_l} \text{GF}(\varphi_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} \text{GF}(\psi_{l,j}))$  such that each  $\varphi_{l,i}$  is an assertion over  $\mathcal{I}$  and each  $\psi_{l,j}$  is an assertion over  $\mathcal{O}$ .

And adding the following games:

- *Persistence to Recurrence (P2R) Games*:  $\bigwedge_{l=1}^k (\bigwedge_{i=1}^{n_l} \text{FG}(\varphi_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} \text{GF}(\psi_{l,j}))$  such that each  $\varphi_{l,i}$  is an assertion over  $\mathcal{I}$  and each  $\psi_{l,j}$  is an assertion over  $\mathcal{O}$ .
- *Recurrence to Persistence (R2P) Games*:  $\bigwedge_{l=1}^k (\bigwedge_{i=1}^{n_l} \text{GF}(\varphi_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} \text{FG}(\psi_{l,j}))$  such that each  $\varphi_{l,i}$  is an assertion over  $\mathcal{I}$  and each  $\psi_{l,j}$  is an assertion over  $\mathcal{O}$ .
- *Persistence to Persistence (P2P) Games*:  $\bigwedge_{l=1}^k (\bigwedge_{i=1}^{n_l} \text{FG}(\varphi_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} \text{FG}(\psi_{l,j}))$  such that each  $\varphi_{l,i}$  is an assertion over  $\mathcal{I}$  and each  $\psi_{l,j}$  is an assertion over  $\mathcal{O}$ .

Note that R2R games are a special case of GR(k) games. In fact in [14] these are called *Separated GR(k) Games*.

A major observation is that in a game played over a separated game structure, the actions of the two players are independent: the environment's actions do not limit the system's actions, and vice versa. In later sections we see how this observation leads to algorithmic improvements in solving separated GR(k) games over a regular GR(k) game.

## VI. SOLVING SEPARATED GAMES

### A. Explicit Solution

### B. Implicit Solution

## VII. SOLVING SEPARATED GAMES

### A. Solving R2R Games

We are given an R2R game that consists of a game structure  $GS$  and a winning condition in a GR(k) form  $\varphi = \bigwedge_{l=1}^k \varphi_l$ , where  $\varphi_l = \bigwedge_{i=1}^{n_l} \text{GF}(a_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} \text{GF}(g_{l,j})$ . Let  $G$  be the game graph of  $GS$ .

Consider an infinite play  $\pi$  in  $GS$ . Like every infinite path on a finite graph,  $\pi$  eventually stabilizes in an SCC  $S$ . Due to separation of variables, the game graph  $G$  can be decomposed into an input graph  $G_{\mathcal{I}}$  and an output graph  $G_{\mathcal{O}}$ . Then the projection of  $S$  on the inputs is an SCC  $S_{\mathcal{I}}$  in  $G_{\mathcal{I}}$ , and the projection of  $S$  on the outputs is an SCC  $S_{\mathcal{O}}$  in  $G_{\mathcal{O}}$ . The input side of  $\pi$  converges to  $S_{\mathcal{I}}$  whereas the output side  $\pi$  converges to  $S_{\mathcal{O}}$ .

Now, let  $S$  be an SCC with projections  $S_{\mathcal{I}}$  on  $G_{\mathcal{I}}$  and  $S_{\mathcal{O}}$  on  $G_{\mathcal{O}}$ .

**Definition 2** (Accepting SCC). *We call  $S$  accepting if for every constraint  $\varphi_l$ , where  $l \in \{1, \dots, k\}$ , one of the following holds:*

**All guarantees hold in  $S$**

*For every  $j \in \{1, \dots, m_l\}$ , there exists  $\text{out} \in S_{\mathcal{O}}$  such that  $\text{out} \models g_{l,j}$ .*

**Some assumption cannot hold in  $S$**

*There exists  $j \in \{1, \dots, n_l\}$  such that for all  $\text{in} \in S_{\mathcal{I}}$ ,  $\text{in} \not\models a_{l,j}$ .*

**Theorem 1.** *The system has a winning strategy in  $(GS, \varphi)$  if and only if it can make sure that every play converges to an accepting SCC, in which all the relevant guarantee states are visited.*

*Proof.* The synthesis procedure to construct such a strategy consists of three steps:

- 1) Synthesize a strategy  $f_B$  for which every play converges to an accepting SCC. The strategy  $f_B$  can be found by synthesizing the weak Büchi game  $(GS, \text{GF}(acc))$ , where  $acc$  is the assertion that accepts exactly those states that belong to accepting SCCs (note that  $(GS, \text{GF}(acc))$  is a well defined weak Büchi game).
- 2) Synthesize a strategy  $f_{travel}$  that travels within every accepting SCC. Crucially, to satisfy the necessary requirements,  $f_{travel}$  constructs a path in  $S_{\mathcal{O}}$  that traverses at least one state satisfying each required  $g_{l,j}$  guarantee.
- 3) Construct an overall winning strategy  $f$  that works as follows: the system plays  $f_B$  until reaching an accepting SCC  $S$ , then the system switches to  $f_{travel}$  to satisfy all required  $g_{l,j}$  guarantees in  $S$ . If the environment moves the play to a non-accepting SCC, the system can start playing  $f_B$  again to reach a different accepting SCC.

A complication arises however when switching between  $f_{travel}$  and  $f_B$ , since it is conceivable that while the system is following  $f_{travel}$ , the environment could move to a different SCC that is outside of the winning region of  $f_B$ . Thus, it is not clear that we can combine these strategies to make an overall winning strategy for the system.

To show that we can indeed combine both strategies, we rely on the following property.

**Lemma 1** (Delay Property). [14] *If  $(i_1, o_1)$  is a state in the winning region of  $f_B$ , and  $(i_2, o_0)$  is a state for which there is a path in  $G_{\mathcal{I}}$  from  $i_1$  to  $i_2$  and a path in  $G_{\mathcal{O}}$  from  $o_0$  to  $o_1$ , then  $(i_2, o_0)$  is also in the winning region of  $f_B$ .*

By the Delay Property, we can guarantee that interleaving  $f_{travel}$  with  $f_B$  remains robust against environment moves, safely keeping the system within a winning region.  $\square$

Here is the fully rewritten and mathematically formalized section in clean LaTeX format. I have included the mandatory source citations in the format '[i]' at the end

of each sentence so you can easily track which source passage the information comes from. You can simply use your LaTeX editor's find-and-replace tool to remove the '[i]' brackets before compiling your final document.

### B. Solving P2Rames

A separated  $P2R$  game differentiates from a regular  $P2R$  game in a separation between the input and output variables in both the game structure and the winning condition. We show in later sections that the separation of variables leads to algorithmic benefits to the synthesis problem.

We are given a Separated  $P2R$  game that consists of a game structure  $GS$  and a winning condition in a  $P2R$  form  $\varphi = \bigwedge_{l=1}^k \varphi_l$ , where each constraint is defined as  $\varphi_l = \bigwedge_{i=1}^{n_l} \text{FG}(a_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} \text{GF}(g_{l,j})$ . Let  $G$  be the game graph of  $GS$ .

Consider an infinite play  $\pi$  in  $GS$ . Like every infinite path on a finite graph,  $\pi$  eventually stabilizes in an SCC  $S$ . Due to the separation of variables, the game graph  $G$  can be decomposed into an input graph  $G_{\mathcal{I}}$  and an output graph  $G_{\mathcal{O}}$ . Consequently, the projection of  $S$  on the inputs is an SCC  $S_{\mathcal{I}}$  in  $G_{\mathcal{I}}$ , and the projection of  $S$  on the outputs is an SCC  $S_{\mathcal{O}}$  in  $G_{\mathcal{O}}$ . The input side of  $\pi$  converges to  $S_{\mathcal{I}}$  whereas the output side  $\pi$  converges to  $S_{\mathcal{O}}$ .

**Definition 3** (Cycles and Assumptions). *Let  $S_{\mathcal{I}}$  be a strongly connected component (SCC), and let  $\text{Cycles}(S_{\mathcal{I}})$  denote the set of all cycles contained in  $S_{\mathcal{I}}$ . For a given cycle  $C \in \text{Cycles}(S_{\mathcal{I}})$  and a boolean expression  $a$ , we say that  $a$  is not maintained in  $C$  (denoted  $a \notin C$ ) if there exists a state  $i \in C$  such that  $i \not\models a$ .*

Now, let  $S$  be an SCC with projections  $S_{\mathcal{I}}$  on  $G_{\mathcal{I}}$  and  $S_{\mathcal{O}}$  on  $G_{\mathcal{O}}$ .

**Definition 4** (Accepting SCC for  $P2R$ ). *We call  $S$  accepting if for every constraint  $\varphi_l$ , where  $l \in \{1, \dots, k\}$ , one of the following two conditions holds:*

**Condition 1: All guarantees hold in  $S$ .** *For every  $j \in \{1, \dots, m_l\}$ , there exists  $o \in S_{\mathcal{O}}$  such that  $o \models g_{l,j}$ .*

**Condition 2:  $S$  cannot maintain all assumptions.**

*Every cycle  $C$  in  $S_{\mathcal{I}}$  contains an assumption  $a_{l,j}$  such that  $a_{l,j} \notin C$ .*

**Algorithmic Verification of Accepting SCCs:** To check whether all guarantees hold in  $S$ , we examine the states of  $S_{\mathcal{O}}$  for each guarantee  $g_{l,j}$  and verify that at least one state satisfies  $g_{l,j}$ . If this is true for every  $g_{l,j}$ , then  $S$  is said to satisfy all guarantees. To determine whether  $S$  cannot maintain all assumptions, we iterate over all assumptions  $a_{l,j}$  and remove from  $S_{\mathcal{I}}$  all states that satisfy the negation  $\neg a_{l,j}$ . Once this filtering has been applied to every assumption, we perform a depth-first search (DFS) on the remaining graph. If no cycle is found, then  $S$  cannot maintain all assumptions; otherwise, the existence of a cycle indicates that  $S$  can maintain them.

**Theorem 2.** *The system has a winning strategy in  $(GS, \varphi)$  if and only if it can force the play to converge to an accepting SCC.*

*Proof.* Following the structure of the  $R2R$  solution, the synthesis procedure relies on two sub-strategies:

- 1) Synthesize a strategy  $f_B$  for which every play converges to an accepting SCC.
- 2) Synthesize a strategy  $f_{\text{travel}}$  that travels within an accepting SCC  $S_{\mathcal{O}}$ . By definition of  $S_{\mathcal{O}}$  being a strongly connected component,  $f_{\text{travel}}$  can be constructed to traverse a path that visits at least one state satisfying each required  $g_{l,j}$  guarantee for constraints where Condition 1 holds.

To prove that combining these strategies forms a winning strategy, we must show that  $f_{\text{travel}}$  wins for the system as long as the play remains within an accepting (good) SCC  $S$ .

Being in an accepting SCC means that for every constraint  $\varphi_l$ , either all guarantees are satisfied in  $S$ , or  $S$  cannot maintain all assumptions.

- If **Condition 1** holds, all guarantees can be satisfied in  $S$ . By traversing the required states in  $S_{\mathcal{O}}$ , the system ensures that every guarantee that can be satisfied within this SCC is eventually covered, fulfilling the  $\text{GF}(g_{l,j})$  requirement.
- If **Condition 2** holds, the guarantees might not all be met, but we know that every cycle  $C$  in  $S_{\mathcal{I}}$  omits some assumption  $a_{l,j}$  (i.e., there exists  $a_{l,j} \notin C$ ). Because the play is restricted to  $S$ , the environment's infinite path must eventually loop entirely within  $S_{\mathcal{I}}$ . Since every cycle in  $S_{\mathcal{I}}$  must include at least one  $\neg a_{l,i}$ , the environment does not have a strategy that can fulfill the premise  $\bigwedge_{i=1}^{n_l} \text{FG}(a_{l,i})$ .

Hence, in either case, the specification  $\varphi_l = \bigwedge_{i=1}^{n_l} \text{FG}(a_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} \text{GF}(g_{l,j})$  is satisfied.  $\square$

A separated  $P2R$  game differentiates from a regular  $P2R$  game in a separation between the input and output variables in both the game structure and winning condition. We show in later sections that the separation of variables leads to algorithmic benefits to the synthesis problem.

We are given a Separated  $P2R$  game that consists of a game structure  $GS$  and a winning condition in a  $P2R$  form  $\varphi = \bigwedge_{l=1}^k \varphi_l$ , where  $\varphi_l = \bigwedge_{i=1}^{n_l} \text{FG}(a_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} \text{GF}(g_{l,j})$ . Let  $G$  be the game graph of  $GS$ . Consider an infinite play  $\pi$  in  $GS$ . Like every infinite path on a finite graph,  $\pi$  eventually stabilizes in an SCC  $S$ . Due to separation of variables, the game graph  $G$  can be decomposed into an input graph  $G_{\mathcal{I}}$  and an output graph  $G_{\mathcal{O}}$ . Then the projection of  $S$  on the inputs is an SCC  $S_{\mathcal{I}}$  in  $G_{\mathcal{I}}$ , and the projection of  $S$  on the outputs is an SCC  $S_{\mathcal{O}}$  in  $G_{\mathcal{O}}$ . The input side of  $\pi$  converges to  $S_{\mathcal{I}}$  whereas the output side  $\pi$  converges to  $S_{\mathcal{O}}$ .

**Definition 5.** Let  $C$  be a cycle in a graph  $G = (V, E)$ . Let  $a$  be a boolean expression. We say that  $a \notin C$  if, for there exists a state  $s \in C$  we have that  $a$  is not satisfied in  $s$ .

**Definition 6.** Let  $S_I$  be a strongly connected component (SCC). We denote by  $Cycles(S_I)$  the set of all cycles contained in  $S_I$ .

Now, let  $S$  be an SCC with projections  $S_I$  on  $G_I$  and  $S_O$  on  $G_O$ . Then we call  $S$  *accepting* if for every constraint  $\varphi_l$ , where  $l \in \{1, \dots, k\}$ , one of the following holds:

**All guarantees hold in  $S$ .** For every  $j \in \{1, \dots, m_l\}$ , there exists  $o \in S_O$  such that  $o \models g_{l,j}$ .

**$S$  cannot maintain all assumptions** Every cycle  $C$  in  $S_I$  contains an assumption  $a_{l,j}$  such that  $a_{l,j} \notin C$ .

To check whether all guarantees hold in  $S$ , we proceed as follows: for each guarantee  $g_{l,j}$ , we examine the states of  $S_O$  and verify that at least one state satisfies  $g_{l,j}$ . If this is true for every  $g_{l,j}$ , then  $S$  is said to satisfy all guarantees. To determine whether  $S$  cannot maintain all assumptions, we proceed as follows: we iterate over all assumptions  $a_{l,j}$  and, for each, remove from  $S_I$  all states that satisfy the negation  $\neg a_{l,j}$ . Once this filtering has been applied to every assumption, we perform a depth-first search (DFS) on the remaining graph. If no cycle is found, then  $S$  cannot maintain all assumptions; otherwise, the existence of a cycle indicates that  $S$  can maintain them.

From here we mimic the structure of the solution of  $R2R$ , construct both:  $f_B$  just the same and  $f_{travel}$  as follows: visit all the states in the current SCC  $S_O$ .

**Claim 1.**  $f_{travel}$  wins for the system as long as it's in a good SCC

*Proof.* This claim follows directly from the definition of a good SCC. Being in a good SCC means that, for every constraint  $\varphi_l$  with  $l \in \{1, \dots, k\}$ , one of the following conditions holds:

- 1) all guarantees are satisfied in  $S$ , or
- 2)  $S$  cannot maintain all assumptions.

By traversing all states in  $S_O$ , the system ensures that every guarantee that can be satisfied within this SCC is eventually covered, fulfilling the  $GF$  requirement. Moreover, for each guarantee that cannot be satisfied in this SCC, we know that every cycle  $C$  in  $S_I$  omits some assumption  $a_{l,j}$ , i.e., there exists  $a_{l,j}$  such that  $a_{l,j} \notin C$ .

Therefore, for every constraint  $i$  in which the walk of  $f_{travel}$  on  $S_O$  does not satisfy  $\bigwedge_{j=1}^{m_l} GF(g_{l,j})$ , we have that the environment does not have a strategy that can fulfill  $\bigwedge_{i=1}^{n_l} FG(a_{l,i})$  since every cycle in  $S_I$  must include at least one  $\neg a_{l,i}$  for some  $l$ .

Hence, in either case, the specification

$$\varphi_l = \bigwedge_{i=1}^{n_l} FG(a_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} GF(g_{l,j})$$

is satisfied.  $\square$

### C. Solving $R2P$ games

A separated  $R2P$  game differentiates from a regular  $R2P$  game in a separation between the input and output variables in both the game structure and the winning condition [6]. We show in later sections that the separation of variables leads to algorithmic benefits to the synthesis problem [6].

We are given a Separated  $R2P$  game that consists of a game structure  $GS$  and a winning condition in an  $R2P$  form  $\varphi = \bigwedge_{l=1}^k \varphi_l$ , where each constraint is defined as  $\varphi_l = \bigwedge_{i=1}^{n_l} GF(a_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} FG(g_{l,j})$  [2, 6]. Let  $G$  be the game graph of  $GS$  [2].

Consider an infinite play  $\pi$  in  $GS$  [2]. Like every infinite path on a finite graph,  $\pi$  eventually stabilizes in an SCC  $S$  [2]. Due to the separation of variables, the game graph  $G$  can be decomposed into an input graph  $G_I$  and an output graph  $G_O$  [2]. Consequently, the projection of  $S$  on the inputs is an SCC  $S_I$  in  $G_I$ , and the projection of  $S$  on the outputs is an SCC  $S_O$  in  $G_O$  [2]. The input side of  $\pi$  converges to  $S_I$  whereas the output side  $\pi$  converges to  $S_O$  [2].

Now, let  $S$  be an SCC with projections  $S_I$  on  $G_I$  and  $S_O$  on  $G_O$  [4].

**Definition 7** (Accepting SCC for  $R2P$ ). We call  $S$  *accepting* if for every constraint  $\varphi_l$ , where  $l \in \{1, \dots, k\}$ , one of the following two conditions holds [4]:

**Condition 1:**  $S$  can maintain all guarantees. There exists a cycle  $C$  in  $S_O$  such that for every state  $o \in C$  and every guarantee  $j \in \{1, \dots, m_l\}$ , we have  $o \models g_{l,j}$ .

**Condition 2:** Some assumption cannot hold in  $S$ . There exists  $i \in \{1, \dots, n_l\}$  such that for all  $i \in S_I$ ,  $i \not\models a_{l,i}$  [1].

**Algorithmic Verification of Accepting SCCs:** To determine whether  $S$  can maintain all guarantees (Condition 1), we iterate over all guarantees  $g_{l,j}$  and remove from  $S_O$  all states that satisfy the negation  $\neg g_{l,j}$  [5]. Once this filtering has been applied for every guarantee associated with  $\varphi_l$ , we perform a depth-first search (DFS) on the remaining graph [5]. If a cycle is found, then  $S$  contains a cycle where all guarantees hold simultaneously [1]; otherwise, the absence of such a cycle indicates that  $S$  cannot maintain them [1]. To check Condition 2, we simply verify if there is any assumption  $a_{l,i}$  for which no state in  $S_I$  satisfies it [1].

**Theorem 3.** The system has a winning strategy in  $(GS, \varphi)$  if and only if it can force the play to converge to an accepting SCC.

*Proof.* Following the established structure, the synthesis procedure relies on two sub-strategies [1]:

- 1) Synthesize a strategy  $f_B$  for which every play converges to an accepting SCC [1].
- 2) Synthesize a strategy  $f_{travel}$  that operates within an accepting SCC  $S_O$  [1]. Crucially, for each constraint

where Condition 1 holds,  $f_{travel}$  guides the system into the specific cycle  $C$  in  $S_{\mathcal{O}}$  where all guarantees hold, and then loops indefinitely within  $C$ , never leaving it.

To prove that combining these strategies forms a winning strategy, we evaluate the game within an accepting SCC  $S$  [7]. For every constraint  $\varphi_l$ , one of the following holds [7]:

- If **Condition 1** holds, the system follows  $f_{travel}$  to reach and indefinitely loop within cycle  $C$ . Because every state in  $C$  satisfies all guarantees  $g_{l,j}$  simultaneously, the system successfully satisfies the persistence requirement  $\bigwedge_{j=1}^{m_l} \text{FG}(g_{l,j})$  [7].
- If **Condition 2** holds, there is some assumption  $a_{l,i}$  that is not satisfied by any state in  $S_{\mathcal{I}}$  [1]. Since the play is confined to  $S$ , the environment's infinite path never visits a state satisfying  $a_{l,i}$ . Therefore, the recurrence requirement  $\text{GF}(a_{l,i})$  fails, rendering the premise of the implication mathematically false [8].

Hence, in either case, the specification

$$\varphi_l = \bigwedge_{i=1}^{n_l} \text{GF}(a_{l,i}) \rightarrow \bigwedge_{j=1}^{m_l} \text{FG}(g_{l,j})$$

is satisfied [8]. Relying on the Delay Property defined previously, we can safely interleave  $f_B$  and  $f_{travel}$  to form the overall winning strategy.  $\square$

## VIII. EVALUATION

- A. Implementation*
- B. Benchmarks*
- C. Competitive Tools*
- D. Experiments*

## IX. CONCLUSION

## REFERENCES

- [1] W. J. Dally and B. Towles, “Route packets, not wires: On-chip interconnection networks,” in *DAC 2001*. Las Vegas, Nevada, USA: ACM, 2001.
- [2] H. Ke, S. Liu, L. Guo, Z. He, L. Song, S. Basalama, Y. Chi, T. Nowatzki, and J. Cong, “Noh: Noc compilation in high-level synthesis,” 2024.
- [3] D. E. Holcomb, B. A. Brady, and S. A. Seshia, “Abstraction-based performance analysis of nocs,” in *Proceedings of the 48th Design Automation Conference (DAC)*. ACM, 2011.
- [4] R. Bloem, B. Jobstmann, N. Piterman, A. Pnueli, and Y. Sa’ar, “Synthesis of reactive(1) designs,” *Journal of Computer and System Sciences*, 2012.
- [5] R. Bloem, S. Galler, B. Jobstmann, N. Piterman, A. Pnueli, and M. Weiglhofer, “Specify, compile, run: Hardware from psl,” *Electronic Notes in Theoretical Computer Science*, vol. 190, pp. 3–16, 2007.
- [6] D. E. Holcomb, “Formal verification and synthesis for quality-of-service in on-chip networks,” Ph.D. dissertation, University of California, Berkeley, 2013.
- [7] D. E. Holcomb and S. A. Seshia, “Compositional performance verification of network-on-chip designs,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2014.
- [8] A. Pnueli, “The temporal logic of programs,” in *Proc. of FOCS*, 1977.
- [9] C. Baier and J. Katoen, *Principles of model checking*. MIT Press, 2008.
- [10] D. Fisman and O. Kupferman, “Reasoning about finite-state switched systems,” in *Haifa Verification Conference*, 2009, pp. 71–86.
- [11] S. B. A. Jr., “Binary decision diagrams,” *IEEE Trans. Computers*, vol. 27, no. 6, pp. 509–516, 1978.
- [12] R. E. Bryant, “Graph-based algorithms for boolean function manipulation,” *IEEE Trans. Computers*, vol. 35, no. 8, pp. 677–691, 1986.
- [13] R. Bloem, B. Jobstmann, N. Piterman, A. Pnueli, and Y. Sa’ar, “Synthesis of reactive(1) designs,” *J. Comput. Syst. Sci.*, vol. 78, no. 3, pp. 911–938, 2012.
- [14] G. Amram, S. Bansal, D. Fried, L. M. Tabajara, M. Y. Vardi, and G. Weiss, “Adapting behaviors via reactive synthesis,” in *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part I*, ser. Lecture Notes in Computer Science, vol. 12759. Springer, 2021, pp. 870–893.
- [15] A. Pnueli and R. Rosner, “On the synthesis of a reactive module,” in *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1989, pp. 179–190.
- [16] R. Bloem, B. Jobstmann, N. Piterman, A. Pnueli, and Y. Sa’ar, “Synthesis of reactive systems,” *Journal of Computer and System Sciences*, vol. 78, no. 3, pp. 911–938, 2012.
- [17] N. Piterman, A. Pnueli, and Y. Sa’ar, “Synthesis of reactive(1) designs,” in *Verification, Model Checking, and Abstract Interpretation*. Springer, 2006, pp. 364–380.
- [18] P. Krishnan, “Synthesis of controllers from specifications given in linear temporal logic,” in *Proceedings of 1994 IEEE International Conference on Robotics and Automation*. IEEE, 1994, pp. 1292–1297.
- [19] B. Alpern and F. B. Schneider, “Defining liveness,” *Information processing letters*, vol. 21, no. 4, pp. 181–185, 1985.
- [20] R. Bloem, B. Jobstmann, N. Piterman, A. Pnueli, and Y. Sa’ar, “Synthesis of reactive(1) designs,” 2012, preprint submitted to Elsevier.
- [21] D. E. Holcomb, “Formal verification and synthesis for quality-of-service in on-chip networks,” Ph.D. dissertation, UC Berkeley, 2013.
- [22] A. B. Kahng, S. K. Kang, and J. Cong, “Clock distribution networks for high-performance microprocessors,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 9, no. 6, pp. 811–823, 2001.
- [23] S. Zhu, G. D. Giacomo, G. Pu, and M. Y. Vardi, “LTLf synthesis with fairness and stability assumptions,” in *Proc. of AAAI*, 2020.
- [24] N. Piterman, A. Pnueli, and Y. Sa’ar, “Synthesis of deterministic Büchi automata from LTL specifications,” in *Proc. of VMCAI*, 2006.
- [25] O. Kupferman, N. Piterman, and M. Y. Vardi, “Safer and abler are better: LTL plus past is more than LTL,” in *Proc. of CONCUR*, 2009.
- [26] G. D. Giacomo and M. Y. Vardi, “Synthesis for LTL on finite traces,” in *Proc. of IJCAI*, 2015.
- [27] F. Dartu and S. M. Nowick, “Metastability in asynchronous circuits: Analysis and design,” *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 20, no. 2, pp. 1–34, 2015.
- [28] T. D. Burd and R. W. Brodersen, “Dynamic voltage and frequency scaling for portable systems,” in *IEEE International Symposium on Low Power Electronics and Design*. IEEE, 2000, pp. 84–89.
- [29] S. Sapatnekar et al., “Runtime timing violation detection for cmos circuits,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 8, pp. 1173–1186, 2011.
- [30] A. Pnueli and R. Rosner, “On the synthesis of a reactive module,” in *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 1989, pp. 179–190.
- [31] A. Pnueli, “The temporal logic of programs,” *18th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 46–57, 1977.
- [32] M. Y. Vardi and P. Wolper, “An automata-theoretic approach to automatic program verification,” in *Proceedings of the First Symposium on Logic in Computer Science (LICS)*. IEEE Computer Society, 1986, pp. 332–344.
- [33] M. Y. Vardi, “An automata-theoretic approach to fair realizability and synthesis,” in *Computer Aided Verification (CAV)*, *Lecture Notes in Computer Science*, P. Wolper, Ed. Springer, 1995, vol. 939, pp. 1–26.
- [34] J. Smith and M. Lee, “Clock stability and timing reliability in high-performance vlsi circuits,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 4, pp. 745–757, 2018.
- [35] X. Wang and A. Patel, “Adaptive clock gating techniques for low-power vlsi designs,” in *Design Automation Conference (DAC)*. IEEE, 2017, pp. 1–6.
- [36] L. Garcia and S. Kim, “Runtime timing monitoring for vlsi circuits,” in *Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2022, pp. 1234–1239.
- [37] J. Kim and D. Thompson, “Synchronizers for metastability resolution in asynchronous circuits,” *IEEE Journal of Solid-State Circuits*, vol. 51, no. 7, pp. 1721–1733, 2016.



- [38] H. Chen and Y. Interian, “A model for generating random quantified boolean formulas,” in *IJCAI-05, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence*, 2005.
- [39] D. Fried, A. Legay, J. Ouaknine, and M. Y. Vardi, “Sequential relational decomposition,” in *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS*, 2018.
- [40] G. Amram, S. Bansal, D. Fried, L. M. Tabajara, M. Y. Vardi, and G. Weiss, “Adapting behaviors via reactive synthesis,” in *Computer Aided Verification - 33rd International Conference, CAV, Proceedings, Part I*, 2021.
- [41] S. Chakraborty, D. Fried, L. M. Tabajara, and M. Y. Vardi, “Functional synthesis via input-output separation,” in *2018 Formal Methods in Computer Aided Design, FMCAD*, 2018.
- [42] C. Ansótegui, M. L. Bonet, J. Giráldez-Cru, and J. Levy, “The fractal dimension of SAT formulas,” in *Automated Reasoning - 7th International Joint Conference, IJCAR. Proceedings*, 2014.
- [43] C. Ansótegui, M. L. Bonet, J. Giráldez-Cru, J. Levy, and L. Simon, “Community structure in industrial SAT instances,” *J. Artif. Intell. Res.*, vol. 66, pp. 443–472, 2019.
- [44] P. Golia, S. Roy, and K. S. Meel, “Manthan: A data-driven approach for boolean function synthesis,” in *Computer Aided Verification - 32nd International Conference, CAV, Proceedings, Part II*, 2020.
- [45] C. Ansótegui, M. L. Bonet, and J. Levy, “On the structure of industrial SAT instances,” in *Principles and Practice of Constraint Programming - CP, 15th International Conference, Proceedings*, 2009.
- [46] S. Akshay, S. Chakraborty, S. Goel, S. Kulal, and S. Shah, “What’s hard about boolean functional synthesis?” in *Computer Aided Verification - 30th International Conference, CAV, Proceedings, Part I*, 2018.
- [47] M. N. Rabe, L. Tentrup, C. Rasmussen, and S. A. Seshia, “Understanding and extending incremental determinization for 2QBF,” in *Computer Aided Verification - 30th International Conference, CAV, Proceedings, Part II*, 2018.
- [48] A. K. John, S. Shah, S. Chakraborty, A. Trivedi, and S. Akshay, “Skolem functions for factored formulas,” in *Formal Methods in Computer-Aided Design, FMCAD*, 2015.
- [49] D. Barak-Pelleg, D. Berend, and J. C. Saunders, “A model of random industrial SAT,” *CoRR*, vol. abs/1908.00089, 2019.
- [50] J. Giráldez-Cru and J. Levy, “A modularity-based random SAT instances generator,” in *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI*, 2015.
- [51] C. Ansótegui, M. L. Bonet, and J. Levy, “Towards industrial-like random SAT instances,” in *IJCAI, Proceedings of the 21st International Joint Conference on Artificial Intelligence*, 2009.
- [52] J. Giráldez-Cru and J. Levy, “Locality in random SAT instances,” in *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI*, 2017.
- [53] S. Akshay, S. Chakraborty, A. K. John, and S. Shah, “Towards parallel boolean functional synthesis,” in *Tools and Algorithms for the Construction and Analysis of Systems - 23rd International Conference, TACAS, Proceedings, Part I*, 2017.
- [54] D. Fried, L. M. Tabajara, and M. Y. Vardi, “Bdd-based boolean functional synthesis,” in *Computer Aided Verification - 28th International Conference, CAV, Proceedings, Part II*, 2016.
- [55] L. M. Tabajara and M. Y. Vardi, “Factored boolean functional synthesis,” in *2017 Formal Methods in Computer Aided Design, FMCAD*, 2017.
- [56] A. Solar-Lezama, “Program sketching,” *Int. J. Softw. Tools Technol. Transf.*, vol. 15, no. 5-6, pp. 475–495, 2013.
- [57] S. Zhu, L. M. Tabajara, J. Li, G. Pu, and M. Y. Vardi, “Symbolic LTLf synthesis,” in *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI*, 2017.
- [58] M. N. Rabe and S. A. Seshia, “Incremental determinization,” in *Theory and Applications of Satisfiability Testing - SAT - 19th International Conference, Proceedings*, 2016.
- [59] J. H. Kukula and T. R. Shiple, “Building circuits from relations,” in *Computer Aided Verification, 12th International Conference, CAV, Proceedings*, 2000.
- [60] M. Barber, “Modularity and community detection in bipartite networks,” *Physical review. E, Statistical, Nonlinear, and Soft Matter Physics*, vol. 76, no. 6 Pt 2, pp. 066 102–066 102, 2007.
- [61] G. Boole, *The Mathematical Analysis of Logic*. Philosophical Library, 1847.
- [62] L. L., “über die auflösung von gleichungen im logischen gebietekalkul,” *Mathematische Annalen*, vol. 69, 1910.
- [63] M. N. Rabe and L. Tentrup, “CAQE: A certifying QBF solver,” in *Formal Methods in Computer-Aided Design, FMCAD*, 2015.
- [64] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, 2008.
- [65] U. Brandes, D. Delling, M. Gaertler, R. Gorke, M. Hoefer, Z. Nikoloski, and D. Wagner, “On modularity clustering,” *IEEE Transactions on Knowledge and Data Engineering*, 2008.
- [66] A. Clauset, M. E. J. Newman, and C. Moore, “Finding community structure in very large networks,” *Physical Review E*, 2004.
- [67] M. E. J. Newman, “Fast algorithm for detecting community structure in networks,” *Physical Review E*, 2004.
- [68] M. E. J. Newman and M. Girvan, “Finding and evaluating community structure in networks,” *Physical Review E*, 2004.
- [69] P. Erdős and A. Rényi, “On random graphs I,” *Publicationes Mathematicae Debrecen*, 1959.
- [70] C. Li, J. Chung, S. Mukherjee, M. Vinyals, N. Fleming, A. Kolokolova, A. Mu, and V. Ganesh, “On the hierarchical community structure of practical boolean formulas,” in *International Conference on Theory and Applications of Satisfiability Testing*, 2021, pp. 359–376.
- [71] G. Amendola, F. Ricca, and M. Truszczynski, “Random models of very hard 2QBF and disjunctive programs: An overview,” in *19th Italian Conference on Theoretical Computer Science*, 2018.